

Estudio de complejidad del sistema

El proyecto implementa un registro de auditoría inmutable basado en un árbol de Merkle. Los eventos se serializan, se convierten en hashes SHA-256 y se organizan como hojas de un árbol binario. La raíz del árbol representa el estado completo del log.

Variables utilizadas

n: número total de eventos registrados. **k**: tamaño medio de la serialización de un evento. En el análisis principal se considera **k** constante respecto a **n**.

Tabla de complejidad

Función	Descripción	Complejidad temporal	Coste espacial
sha256(data)	Calcula el hash de una cadena	$O(k)$	$O(1)$
EventoLog.serializar()	Convierte un evento a texto	$O(k)$	$O(k)$
NodoMerkle.calcular_hash()	Calcula hash de hoja o nodo interno	$O(k)$ en hoja / $O(1)$ interno	$O(1)$
insertar_evento(evento)	Añade un evento y reconstruye el árbol	$O(n)$	$O(n)$
construir_arbol()	Crea hojas y construye la raíz	$O(n)$	$O(n)$
_construir_recursoivo(nodos)	Combina nodos por parejas hasta la raíz	$O(n)$	$O(n)$
get_raiz_hash()	Devuelve el hash raíz almacenado	$O(1)$	$O(1)$
recalcular_raiz_hash()	Reconstruye el árbol desde eventos actuales	$O(n)$	$O(n)$
verificar_integridad()	Compara raíz almacenada y recalculada	$O(n)$	$O(n)$
obtener_prueba(indice)	Genera hashes hermanos de una hoja	$O(\log n)$	$O(\log n)$
verificar_evento(indice)	Reconstruye la raíz desde una prueba	$O(\log n)$	$O(\log n)$

Análisis teórico

La construcción del árbol es lineal porque cada evento genera una hoja y los nodos internos se crean combinando parejas de hashes. En total, el número de nodos procesados es proporcional a **n**.

La verificación completa también es $O(n)$, ya que recalcula la raíz desde todos los eventos actuales para compararla con el hash sellado. Esta operación permite detectar modificaciones directas sobre los logs.

Las pruebas Merkle tienen coste $O(\log n)$ porque solo se necesita un hash hermano por cada nivel del árbol. En un árbol equilibrado, la altura crece logarítmicamente respecto al número de eventos.

Mejor, promedio y peor caso

Operación	Mejor caso	Caso promedio	Peor caso
Construcción del árbol	$O(n)$	$O(n)$	$O(n)$
Verificación completa	$O(n)$	$O(n)$	$O(n)$
Obtención de prueba	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash raíz	$O(1)$	$O(1)$	$O(1)$

Tiempos medios esperados

Para volúmenes pequeños o medianos de logs, el coste lineal de reconstrucción es aceptable. Si el sistema creciera hasta millones de eventos, sería mejor actualizar solo la rama afectada del árbol y persistir los nodos, reduciendo el coste de inserción a $O(\log n)$.

Propuestas de mejora

1. Mantener referencias a hojas para actualizar ramas sin reconstruir todo el árbol.
2. Guardar el hash raíz en un servidor externo o repositorio inmutable.
3. Persistir eventos y nodos en base de datos para trabajar con logs grandes.
4. Añadir firmas digitales a los informes de auditoría.